Networking Basics: IP, Subnet, Network ID and Broadcast

Simple notes for students, in a proper order, with examples, tricks and Linux commands

Part	Topic
1	What is networking? LAN, MAN, WAN
2	IP address, host, network, subnet, broadcast
3	IPv4 format and converting an IP to binary
4	IPv4 vs IPv6
5	IP classes, identifying a class, loopback, private and public
6	Subnet mask: what it is and how to calculate it
7	Network ID, host ID and broadcast ID: how to calculate
8	Subnetting: splitting a network into subnets
9	Gateway, DNS and the full journey of a web request
10	Linux commands, tricks, cheat sheet, practice questions

1. What is Networking?

Networking means connecting computers and devices so they can **communicate and share data**. Examples: your laptop opens a website, a server connects to a database, two servers talk inside a Kubernetes cluster.

Network	Full form	Area covered	Example
LAN	Local Area Network	Small: home, office, school	Office Wi-Fi, college lab
MAN	Metropolitan Area Network	One city	Company offices across a city
WAN	Wide Area Network	Country or world	The Internet

Trick: LAN = one building. **MAN** = one city. **WAN** = many cities or countries. The Internet is the biggest WAN.

2. IP Address, Host, Network, Subnet, Broadcast

Term	Simple meaning	Analogy
IP address	Unique number of a device on a network. Data is sent to it.	Your home address

Term	Simple meaning	Analogy
Host	Any device with an IP address: PC, phone, server, printer.	A house
Network	A group of hosts that can talk directly. They share the same network part.	A street
Subnet	A smaller network made by splitting a bigger one.	A lane in the street
Subnet mask	Tells which part of the IP is the network and which is the host.	Where the street name ends
Network ID	The first address of a network. It names the network. Not given to a device.	Street name
Host ID	The part of the IP that identifies the device inside the network.	House number
Broadcast address	The last address of a network. A message sent here reaches all hosts.	Loudspeaker for the street
Gateway	The router that connects your network to other networks and the Internet.	The street's exit gate

```
IP address  : 192.168.1.25
Subnet mask : 255.255.255.0

Network part: 192.168.1      (same for all hosts in this network)
Host part   : 25             (different for every device)

Network ID  : 192.168.1.0    (first address)
Broadcast   : 192.168.1.255  (last address)
Hosts       : 192.168.1.1 to 192.168.1.254
```

3. IPv4 Format and IP to Binary

An **IPv4** address has **32 bits**, written as **4 octets** (numbers 0 to 255) separated by dots. Each octet is **8 bits** (4 x 8 = 32). Computers work in binary, so we must be able to convert.

Decimal to binary: the 128-64-32 table

Write the bit values **128 64 32 16 8 4 2 1**. Starting from the left, put **1** if the value fits into what is left, otherwise **0**.

Decimal	128	64	32	16	8	4	2	1	Binary
192	1	1	0	0	0	0	0	0	11000000
168	1	0	1	0	1	0	0	0	10101000
1	0	0	0	0	0	0	0	1	00000001
25	0	0	0	1	1	0	0	1	00011001
255	1	1	1	1	1	1	1	1	11111111

Worked example: 168. 168 - 128 = 40 (bit 128 = 1). 64 does not fit in 40 (0). 32 fits: 40 - 32 = 8 (1). 16 no (0). 8 fits: 8 - 8 = 0 (1). The rest 0. Result: **10101000**.

Full IP to binary

```
IP        : 192      .  168      .  1        .  25
Binary    : 11000000 .  10101000 .  00000001 .  00011001

So 192.168.1.25 = 11000000.10101000.00000001.00011001
```

Binary to decimal

Add the bit values where there is a 1. Example: **11001000** = 128 + 64 + 8 = **200**.

> **Trick:** Memorise the 8 values **128 64 32 16 8 4 2 1**. Any octet that is all 1s = 255, all 0s = 0. Numbers like 128, 192, 224, 240, 248, 252, 254 are the only possible **mask** values (1s on the left, 0s on the right).

4. IPv4 vs IPv6

Point	IPv4	IPv6
Size	32 bits	128 bits
Written as	4 decimal octets: 192.168.1.10	8 groups of hex: 2001:db8:85a3::8a2e:370:7334
Separator	Dot (.)	Colon (:)
Total addresses	About 4.3 billion	About 3.4 x 10^{38} (practically unlimited)
Broadcast	Yes	No (uses multicast)
NAT needed?	Yes, because addresses ran short	Not needed
Loopback	127.0.0.1	::1
Example	8.8.8.8	2001:4860:4860::8888

Shortening an IPv6 address (2 rules)

```
Full     : 2001:0db8:85a3:0000:0000:8a2e:0370:7334
Rule 1   : drop leading zeros in each group
           2001:db8:85a3:0:0:8a2e:370:7334
Rule 2   : replace ONE run of all-zero groups with ::
Short    : 2001:db8:85a3::8a2e:370:7334
```

5. IP Classes, Loopback, Private and Public

In the classful system, the **first octet** tells the class. The class decides the default mask: how much is network and how much is host.

Class	First octet	Starts with bits	Default mask	Network | Host	Hosts per network
A	1 - 126	0	255.0.0.0 (/8)	N.H.H.H	16,777,214
B	128 - 191	10	255.255.0.0 (/16)	N.N.H.H	65,534
C	192 - 223	110	255.255.255.0 (/24)	N.N.N.H	254
D	224 - 239	1110	none	-	Multicast
E	240 - 255	1111	none	-	Experimental

Trick: Look **only at the first number**. **1-126 = A, 128-191 = B, 192-223 = C.** Remember: A is the first, so it has the **smallest numbers** and the **most hosts**.

Loopback address: 127.0.0.1

The whole range **127.0.0.0 to 127.255.255.255** is **loopback**. Data sent to it never leaves your computer. The usual address is **127.0.0.1**, named **localhost**. It is used to test the network software and local servers. This is why Class A is 1 to **126**.

How to identify the class: examples

IP address	First octet	Class	Default mask	Note
10.5.4.3	10	A	255.0.0.0	Private
8.8.8.8	8	A	255.0.0.0	Public (Google DNS)
127.0.0.1	127	-	-	Loopback
150.20.5.9	150	B	255.255.0.0	Public
172.16.5.20	172	B	255.255.0.0	Private
192.168.1.10	192	C	255.255.255.0	Private
200.10.5.7	200	C	255.255.255.0	Public
224.0.0.5	224	D	-	Multicast
250.1.1.1	250	E	-	Experimental

Private and public ranges

Type	Range	Class
Private	10.0.0.0 to 10.255.255.255	A
Private	172.16.0.0 to 172.31.255.255	B
Private	192.168.0.0 to 192.168.255.255	C
Public	Everything else (e.g. 8.8.8.8, 142.250.183.14)	A, B, C

Remember: Private addresses are used inside homes and offices and cannot be reached from the Internet. The router uses **NAT** to share one public IP among all private devices. Warning: **172.32.x.x is public**, because private Class B stops at 172.31.

6. Subnet Mask: What and How to Calculate

A **subnet mask** is a 32-bit number made of **1s on the left** (network part) and **0s on the right** (host part). It is written as 255.255.255.0 or as **/24** (the number of 1 bits). **/24 means: the first 24 bits are the network.**

```
/24 = 11111111.11111111.11111111.00000000 = 255.255.255.0
/16 = 11111111.11111111.00000000.00000000 = 255.255.0.0
/8  = 11111111.00000000.00000000.00000000 = 255.0.0.0
```

Method 1: CIDR (/n) to mask

Write **n** ones, then zeros to make 32 bits, and convert each octet. **Example: /26** = 26 ones = 11111111.11111111.11111111.**11**000000 = 255.255.255.**192** (128 + 64 = 192).

Method 2: mask to CIDR

Count the 1 bits. **Example: 255.255.255.224** = 8 + 8 + 8 + 3 (224 = 11100000) = **/27**.

Mask values to memorise (last octet)

Prefix	Mask	Binary of last octet	Host bits	Usable hosts	Block size
/24	255.255.255.0	00000000	8	254	256
/25	255.255.255.128	10000000	7	126	128
/26	255.255.255.192	11000000	6	62	64
/27	255.255.255.224	11100000	5	30	32
/28	255.255.255.240	11110000	4	14	16
/29	255.255.255.248	11111000	3	6	8
/30	255.255.255.252	11111100	2	2	4

Trick: Block size = 256 - last mask octet. Example: mask .192 gives 256 - 192 = **64**. **Usable hosts = block size - 2.** (64 - 2 = 62.) **Total addresses = $2^{\text{host bits}}$.**

7. Network ID, Host ID and Broadcast ID: How to Calculate

Do these steps in order. **Rule:** Network ID = host bits all **0**. Broadcast = host bits all **1**. First host = Network ID + 1. Last host = Broadcast - 1.

Step	What to do
1	Find the class from the first octet (or read the /n given).
2	Write the mask and find the network part and host part.
3	Network ID: set all host bits to 0 (the AND method).
4	Broadcast: set all host bits to 1.
5	Hosts: from Network ID + 1 to Broadcast - 1.
6	Number of hosts = $2^{\text{host bits}} - 2$.

Example 1 (easy): 192.168.1.25 with 255.255.255.0

Step	Result
Class	C (192). Mask /24: network part 192.168.1, host part 25
Network ID	192.168.1.0 (host part set to 0)
Broadcast	192.168.1.255 (host part set to 255 = all 1s)
Hosts	192.168.1.1 to 192.168.1.254 = 254 hosts

Example 2: 172.16.5.20 with 255.255.0.0 (/16)

Step	Result
Class	B. Network part 172.16, host part 5.20
Network ID	172.16.0.0
Broadcast	172.16.255.255
Hosts	172.16.0.1 to 172.16.255.254 = 65,534 hosts

Example 3 (binary AND method): 192.168.1.100 with 255.255.255.192 (/26)

Only the last octet changes. AND each bit: **1 AND 1 = 1**, anything else = 0.

```
IP   last octet  100 = 01100100
Mask last octet  192 = 11000000
--------------------------------
AND (Network)        = 01000000 = 64

Host bits (last 6) set to 1 -> 01111111 = 127 (Broadcast)
```

Result	Value
Network ID	192.168.1.64
Broadcast	192.168.1.127
Usable hosts	192.168.1.65 to 192.168.1.126 = 62 hosts

The fast block-size trick (no binary needed)

```
IP 192.168.1.100 /26  -> mask .192 -> block size = 256 - 192 = 64
List the blocks:  0, 64, 128, 192
100 lies between 64 and 128  -> Network ID = .64
Broadcast = next block - 1   -> 128 - 1 = .127
Hosts = .65 to .126
```

Trick: Network ID = the largest block start that is **less than or equal to** the host number. **Broadcast** = the next block start **minus 1**.

Example 4: 10.0.0.77 with 255.255.255.240 (/28)

Step	Result
Block size	256 - 240 = 16. Blocks: 0, 16, 32, 48, 64, 80 ...
77 lies between	64 and 80
Network ID	10.0.0.64
Broadcast	10.0.0.79 (80 - 1)
Hosts	10.0.0.65 to 10.0.0.78 = 14 hosts

8. Subnetting: Splitting a Network

Subnetting splits one big network into smaller ones by **borrowing bits from the host part** and giving them to the network part. This saves addresses, improves security and reduces traffic.

You need	Formula	Then
S subnets	Borrow n bits so that 2^n is at least S	New prefix = old prefix + n
H hosts per subnet	Keep m host bits so that 2^m - 2 is at least H	New prefix = 32 - m

Example A: split 192.168.1.0/24 into 4 equal subnets

Need 4 subnets: $2^2 = 4$, so borrow **2 bits**. New prefix = 24 + 2 = **/26**, mask **255.255.255.192**, block size 64, 62 hosts each.

Subnet	Network ID	Usable hosts	Broadcast
1	192.168.1.0/26	192.168.1.1 - .62	192.168.1.63
2	192.168.1.64/26	192.168.1.65 - .126	192.168.1.127
3	192.168.1.128/26	192.168.1.129 - .190	192.168.1.191
4	192.168.1.192/26	192.168.1.193 - .254	192.168.1.255

Example B: a department needs 50 hosts

2^6 - 2 = 62, which is at least 50, so keep **6 host bits**. Prefix = 32 - 6 = **/26**. (2^5 - 2 = 30 is too small.)

Trick: Quick table: **/25 = 2 subnets, /26 = 4, /27 = 8, /28 = 16, /29 = 32, /30 = 64** (from a /24). Each step **doubles** the subnets and **halves** the hosts.

9. Gateway, DNS and the Journey of a Web Request

Component	Purpose	Linux command
IP address	Identifies a device	ip addr
Subnet mask	Separates network ID and host ID	ip addr (the /24)
Gateway	Sends traffic to other networks	ip route
DNS	Turns names (google.com) into IPs	dig google.com

What happens when you open https://google.com

Step	What happens
1	Browser asks the DNS server: 'What is the IP of google.com?'
2	DNS replies with the IP, for example 142.250.183.14.
3	Your computer checks: is this IP in my local network (using the mask)? No.
4	The packet is sent to the **default gateway** (router, e.g. 192.168.1.1).
5	The gateway forwards it through the Internet to Google's server.
6	Google sends the web page back along the same path.

```
PC 192.168.1.10 --> DNS --> IP found --> Gateway 192.168.1.1 --> Internet --> Google
```

Remember: Without a gateway, your computer can talk only to devices in the **same network**. The mask is how your computer decides 'same network' or 'send to the gateway'.

10. Linux Commands to Show Students

Syntax	Meaning
`ip addr / ip -4 addr show`	Show IP addresses and masks (the inet lines).
`hostname -I`	Show the system's IP address.
`ip route`	Show routes and the default gateway.
`ping -c 3 HOST`	Test connectivity (-c 3 = three packets).
`dig NAME / nslookup NAME`	Ask DNS for the IP of a name.
`curl -I https://google.com`	Test web (HTTP/HTTPS) access, headers only.

COPY AND PASTE: Network commands

```
ip -4 addr show
hostname -I
ip route
ping -c 3 127.0.0.1
ping -c 3 google.com
nslookup google.com
curl -sI https://google.com | head -1
```

Command	What it does (in simple words)
`ip -4 addr show`	IPv4 address and mask of each network card.
`hostname -I`	Only the IP addresses.
`ip route`	The line starting with default via shows the gateway.
`ping -c 3 127.0.0.1`	Pings yourself (loopback): proves the network software works.
`ping -c 3 google.com`	Tests DNS and Internet at once (needs Internet).
`nslookup google.com`	Shows the IP that DNS returns.
`curl -sI https://google.com | head -1`	Shows the first line of the web reply, e.g. HTTP/2 301.

Output (what students will see)

```
    inet 192.168.1.10/24 brd 192.168.1.255 scope global eth0 <-- IP/mask
192.168.1.10                          <-- hostname -I
default via 192.168.1.1 dev eth0      <-- the gateway
3 packets transmitted, 3 received, 0% packet loss   <-- ping OK
Name: google.com   Address: 142.250.183.14        <-- DNS answer
```

Check your calculations with Python (already on most Linux systems)

COPY AND PASTE: Network ID, broadcast and binary calculator

```
python3 << 'EOF'
import ipaddress
n = ipaddress.ip_interface('192.168.1.100/26').network
print(n, n.broadcast_address, n.netmask, n.num_addresses)
print('.'.join(format(int(o), '08b') for o in '192.168.1.10'.split('.')))
EOF
```

Command	What it does (in simple words)
`n = ipaddress.ip_interface(...).network`	Calculates the network of any IP/prefix. Change 192.168.1.100/26 to test your own answers.
`print(n, n.broadcast_address, ...)`	Prints network ID, broadcast, mask and total addresses.
`print('.'.join(format(int(o), '08b') ...))`	Converts each octet of an IP into 8-bit binary.

Output (what students will see)

```
192.168.1.64/26 192.168.1.127 255.255.255.192 64   <-- net, bcast, mask, total
11000000.10101000.00000001.00001010   <-- 192.168.1.10 in binary
```

11. Cheat Sheet and Practice

Topic	Remember
Octet / IPv4 size	8 bits each, 4 octets = 32 bits
Bit values	128 64 32 16 8 4 2 1
Class A / B / C	1-126 / 128-191 / 192-223; masks /8, /16, /24
Class D / E	224-239 multicast / 240-255 experimental
Loopback	127.0.0.1 (IPv6: ::1)
Private ranges	10.x.x.x, 172.16-31.x.x, 192.168.x.x
Network ID / Broadcast	Host bits all 0 / host bits all 1
Block size	256 - last mask octet
Usable hosts	2 to the power of host bits, minus 2
IPv4 / IPv6	32 bits dotted decimal / 128 bits hex with colons

Practice questions

- Convert 172.16.10.5 to binary.
- Identify the class and default mask of 130.45.6.7 and 199.1.1.1.
- Is 172.32.5.5 private? Is 192.168.100.1 private?
- Convert /27 to a subnet mask. How many usable hosts?
- Find the network ID, broadcast and host range of 192.168.10.200/26.
- Find the network ID and broadcast of 10.1.1.130/25.
- Split 192.168.5.0/24 into 8 subnets. Give the new mask and block size.
- Write 2001:0db8:0000:0000:0000:ff00:0042:8329 in short form.

Answers

No.	Answer
1	10101100.00010000.00001010.00000101 (172=10101100, 16=00010000, 10=00001010, 5=00000101)
2	130.45.6.7 = Class B, 255.255.0.0. 199.1.1.1 = Class C, 255.255.255.0.
3	172.32.5.5 is public (private stops at 172.31). 192.168.100.1 is private.
4	/27 = 255.255.255.224, block size 32, 30 usable hosts.
5	Mask .192, block 64: 200 is in 192-255. Network 192.168.10.192, broadcast 192.168.10.255, hosts .193 to .254 (62).

No.	Answer
6	/25 = mask .128, block 128: 130 is in 128-255. Network 10.1.1.128, broadcast 10.1.1.255.
7	8 subnets = 3 bits, so /27, mask 255.255.255.224, block size 32 (0, 32, 64 ... 224), 30 hosts each.
8	2001:db8::ff00:42:8329

Summary: Conclusion: Learn the 8 bit values and the mask table. Then every question (class, mask, network ID, broadcast, hosts, subnets) can be solved quickly with the block-size trick.